Chapter 1 Solutions

Computer Networking: A Top-Down Approach, 9th Edition

Computer Networks and the Internet — Problems P1–P30

Complete, step-by-step worked solutions with all derivations, formulas, and final answers

Prepared for Codecrafter

July 9, 2026

Contents

1 ATM–Bank Application-Layer Protocol 2

2 Generalizing End-to-End Delay for P Back-to-Back Packets 4

3 Circuit Switching vs. Packet Switching for Steady-Rate Traffic 5

4 Circuit-Switched Ring Network (Switches A, B, C, D) 6

5 Car-Caravan Analogy 8

6 Propagation Delay and Transmission Delay Fundamentals 9

7 VoIP End-to-End Delay 11

8 Circuit Switching vs. Packet Switching: Shared 10 Mbps Link 12

9 Circuit Switching on a 1 Gbps Link 14

10 End-to-End Delay Through Two Packet Switches 15

11 End-to-End Delay With Cut-Through Switching 16

12 Queuing Delay Behind Packets Already in the Buffer 17

13 Average Queuing Delay for a Burst of N Packets 18

14 Total Delay as a Function of Traffic Intensity 19

15 Total Delay in Terms of Arrival and Service Rates 20

16 Little's Formula 21

17 Generalizing Equation 1.2 for Heterogeneous Nodes 22

18 Hands-On Traceroute Exercise 23

19 Metcalfe's Law 24

20 Throughput With M Client-Server Pairs 25

21 Maximum Throughput Over M Parallel Paths 26

22 Probability of Successful Delivery and Expected Retransmissions 27

23 Packet-Pair Bottleneck-Bandwidth Probing 28

24 50 TB Boston-to-LA: Dedicated Link vs. FedEx 30

25 Bandwidth-Delay Product (R = 5 Mbps) 31

26 Choosing R So a Bit's Width Equals the Link Length **33**

27 Problem P25 Repeated with R = 500 Mbps **34**

28 Continuous vs. Stop-and-Wait Transfer Time **35**

29 Geostationary Satellite Microwave Link **36**

30 Layering: Headers in the Airline Analogy **37**

Problem 1 ATM–Bank Application-Layer Protocol

Problem Statement

Design an application-layer protocol between an ATM and a bank's centralized computer that verifies a card and password, allows an account balance query, and allows a withdrawal (handling the case of insufficient funds). List the messages exchanged and the actions taken on transmission/receipt. Sketch a simple, error-free withdrawal using a diagram similar to Figure 1.2. State the assumptions made about the underlying end-to-end transport service.

Solution

Protocol entities and roles. There are two application-layer entities: the *ATM client* and the *Bank server*. The ATM initiates every exchange (a request–reply, client–server style protocol, just like Figure 1.2 in the text).

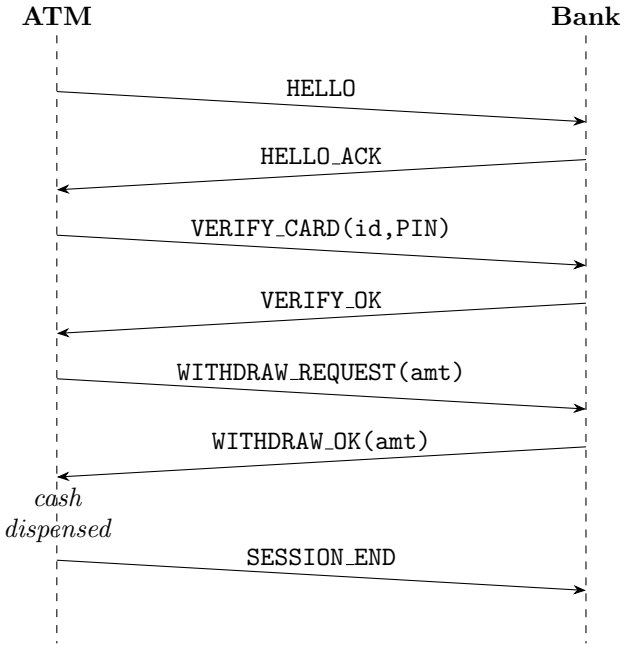
Messages defined by the protocol:

1. `HELLO` — sent by the ATM to open a session with the bank server.
2. `HELLO_ACK` — sent by the bank to confirm the session is open.
3. `VERIFY_CARD(cardID, PIN)` — sent by the ATM once a card is inserted and a PIN is entered.
4. `VERIFY_OK` / `VERIFY_FAIL` — sent by the bank after checking the card number and PIN against its account database.
5. `BALANCE_REQUEST(accountID)` — sent by the ATM if the user requests a balance inquiry.
6. `BALANCE_REPLY(amount)` — sent by the bank in response.
7. `WITHDRAW_REQUEST(accountID, amount)` — sent by the ATM if the user requests cash.
8. `WITHDRAW_OK(amount)` — sent by the bank *only if* the account balance $\geq$ amount; the bank debits the account *before* replying.
9. `WITHDRAW_DENIED(reason)` — sent by the bank if funds are insufficient; no debit occurs.
10. `SESSION_END` — sent by the ATM when the user is finished (e.g., card ejected).

Actions on transmission/receipt.

- **ATM**, on card insertion: reads the card's account ID, prompts for a PIN, and sends `VERIFY_CARD`. It then blocks until it receives `VERIFY_OK` or `VERIFY_FAIL` (or a transport-layer timeout).
- **Bank**, on receipt of `VERIFY_CARD`: looks up the account record, compares the PIN (or its hash) to the value on file, and replies `VERIFY_OK` or `VERIFY_FAIL`.
- **ATM**, on receipt of `VERIFY_OK`: displays the menu; if the user selects "withdraw," it sends `WITHDRAW_REQUEST` with the requested amount.
- **Bank**, on receipt of `WITHDRAW_REQUEST`: checks balance $\geq$ amount. If true, it atomically debits the account and replies `WITHDRAW_OK`; otherwise it replies `WITHDRAW_DENIED`.
- **ATM**, on receipt of `WITHDRAW_OK`: dispenses the cash, prints a receipt, and eventually sends `SESSION_END`. On receipt of `WITHDRAW_DENIED`: displays an error message and dispenses nothing.

Sequence diagram for an error-free withdrawal (time flows downward, as in Figure 1.2):



Assumptions about the underlying end-to-end transport service. The protocol assumes the transport layer:

- is *reliable* — no message is lost, corrupted, or duplicated (critical, since a lost `WITHDRAW_OK` must never cause a double debit, and a lost `WITHDRAW_REQUEST` must never dispense cash without a debit);
- delivers messages *in order*, so the ATM and bank can rely on strict request/reply sequencing;
- is *connection-oriented*, so a session persists for the duration of the transaction and either side can detect if the connection drops (e.g., via a timeout), at which point the ATM aborts the transaction and returns the card rather than dispensing cash on an unconfirmed reply;
- provides basic data integrity/confidentiality for the PIN in transit (in practice this is layered on top via encryption, but the application protocol assumes the channel does not leak the PIN in the clear).

Problem 2 Generalizing End-to-End Delay for P Back-to-Back Packets

Problem Statement

Equation 1.1 gives the end-to-end delay for sending one packet of length L over N links of transmission rate R (store-and-forward, no propagation/queuing/processing delay). Generalize this formula for sending P such packets back-to-back over the N links.

Solution

Equation 1.1 states that for a single packet,

$$d_{\text{end-to-end}} = N \cdot \frac{L}{R}.$$

Each store-and-forward switch must receive an *entire* packet before forwarding any bit of it, so a single packet incurs a transmission delay of L/R at each of the N links in sequence.

Now suppose P packets, each of length L, are sent back-to-back (i.e., the source begins transmitting packet $k+1$ the instant it finishes transmitting packet k). The *last* bit of the *last* (P^{th}) packet determines the total delay:

- The source needs $P \cdot (L/R)$ seconds just to push all P packets, back-to-back, onto the first link. The last bit of the Pth packet leaves the source at $t = P \cdot (L/R)$ and arrives at the first switch at that same time (ignoring propagation delay, per the problem's convention).
- From there, the Pth packet alone must still be store-and-forwarded across the remaining $N-1$ links, each contributing L/R.

Hence,

$$d_{\text{end-to-end}} = P \cdot \frac{L}{R} + (N-1) \cdot \frac{L}{R}$$

Final Answer

$$\boxed{d_{\text{end-to-end}} = (N + P - 1)\,\frac{L}{R}}$$

Sanity check: for $P = 1$ this reduces exactly to Equation 1.1, $d = N(L/R)$.

Problem 3 Circuit Switching vs. Packet Switching for Steady-Rate Traffic

Problem Statement

An application sends data at a steady rate (an N-bit unit every k ms, k small and fixed) for a long period of time. **(a)** Would a packet-switched or circuit-switched network be more appropriate? Why? **(b)** If packet switching is used, and the sum of all such applications' data rates is always less than the capacity of every link, is congestion control needed? Why?

Solution

(a) A **circuit-switched** network is the more natural fit. Circuit switching reserves a constant amount of bandwidth (or time slots) along the entire path for the duration of the connection. Because this application's demand is *steady and predictable* and lasts a *long time*, a dedicated circuit is not wasted the way it would be for bursty traffic – the app will continuously use essentially all of what it reserves. Circuit switching also gives the application a smooth, guaranteed, jitter-free delay, which matches a steady real-time-like data stream well. Packet switching's chief advantage – statistical multiplexing among many bursty sources – provides little extra benefit here, while it does add variable queuing delay and per-packet header overhead that a steady stream doesn't need.

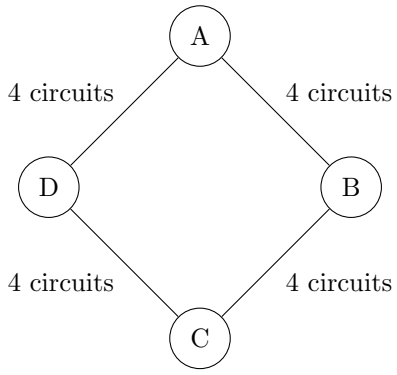
(b) **No significant congestion control is needed** in this idealized scenario. Congestion control exists to keep the aggregate offered load from exceeding link capacity (which causes growing queues, delay, and loss). Here we are told that the sum of the applications' data rates is *always* less than the capacity of *every* link in the network. Since demand never exceeds supply on any link, queues cannot build up without bound, so the mechanisms that a general-purpose congestion-control scheme would provide are unnecessary for this particular, well-behaved traffic mix.

Problem 4 Circuit-Switched Ring Network (Switches A, B, C, D)

Problem Statement

A circuit-switched ring network has four switches A, B, C, D (clockwise), with four circuits on each link. **(a)** What is the maximum number of simultaneous connections that can be in progress? **(b)** If all connections are between A and C, what is the maximum number of simultaneous connections? **(c)** Can four A–C connections and four B–D connections all be routed simultaneously through the four links?

Solution



(a) There are 4 links (A–B, B–C, C–D, D–A), each carrying 4 independent circuits, giving $4 \times 4 = 16$ total circuit-slots in the network. Every connection needs at least 1 circuit-slot on at least 1 link, and a *direct* (single-hop, adjacent-switch) connection needs exactly 1 slot on exactly 1 link – the most resource-efficient possible connection. Filling every slot with a direct neighbor-to-neighbor connection (4 on A–B, 4 on B–C, 4 on C–D, 4 on D–A) uses all 16 slots and is achievable with no conflicts.

Final Answer

Maximum simultaneous connections = 4 links × 4 circuits/link = **16**.

(b) A and C are not adjacent, so every A–C connection must be routed over *two* links: either via B (using link A–B and link B–C) or via D (using link A–D and link D–C). A connection routed via B consumes one circuit on *both* A–B and B–C, so the number of such connections is limited by $\min(4, 4) = 4$. Likewise, at most 4 connections can be routed via D. Since the "via B" path and the "via D" path share no links, both can be used at full capacity simultaneously.

Final Answer

Maximum simultaneous A–C connections = 4 (via B) + 4 (via D) = **8**.

(c) **Yes.** Split each group of four connections evenly between the two routes around the ring:

- 2 of the A–C connections routed via B (using links A–B, B–C)
- 2 of the A–C connections routed via D (using links A–D, D–C)

- 2 of the B–D connections routed via A (using links B–A, A–D)
- 2 of the B–D connections routed via C (using links B–C, C–D)

Now tally the load on each link:

$$\text{A–B} : 2 + 2 = 4, \quad \text{B–C} : 2 + 2 = 4, \quad \text{C–D} : 2 + 2 = 4, \quad \text{D–A} : 2 + 2 = 4.$$

Final Answer

Every link is loaded to exactly its 4-circuit capacity, so **yes**, all eight connections (four A–C and four B–D) can be routed simultaneously – by sending half of each group clockwise and half counter-clockwise around the ring.

Problem 5 Car-Caravan Analogy

Problem Statement

Using the car-caravan analogy of Section 1.4 with a propagation speed of 100 km/hour: **(a)** the caravan travels 175 km, starting in front of one tollbooth, passing through a second, and finishing just after a third – find the end-to-end delay. **(b)** Repeat with eight cars instead of ten.

Solution

Stated assumptions (carried over from the original Section 1.4 caravan example, which this problem builds on): the caravan has $N = 10$ cars, each tollbooth services (i.e., "transmits") cars one at a time at a rate of 12 seconds/car, and the three tollbooths are evenly spaced along the 175 km route – 87.5 km between tollbooth 1 and 2, and 87.5 km between tollbooth 2 and 3.

This is exactly analogous to a packet (the caravan) being *store-and-forwarded* through two intermediate "switches" (tollbooths 2 and 3), with an initial "transmission" event at tollbooth 1:

- **Toll (transmission) delay** at one booth for all N cars, serviced sequentially: $N \times 12$ sec. This occurs at *each* of the 3 tollbooths encountered $\Rightarrow$ total toll delay $= 3N(12)$.
- **Propagation delay** for one 87.5 km segment: $d = \dfrac{87.5 \text{ km}}{100 \text{ km/hr}} = 0.875 \text{ hr} = 3150 \text{ sec}$. There are 2 such segments (booth1$\rightarrow$booth2, booth2$\rightarrow$booth3) $\Rightarrow$ total propagation delay $= 2(3150) = 6300$ sec.

(a) $N = 10$ **cars:**

$$\text{Toll delay} = 3 \times 10 \times 12 = 360 \text{ sec}$$

$$d_{\text{total}} = 360 + 6300 = 6660 \text{ sec}$$

Final Answer

$d_{\text{total}} = 6660$ s $= 111$ minutes $= 1$ hr 51 min.

(b) $N = 8$ **cars:**

$$\text{Toll delay} = 3 \times 8 \times 12 = 288 \text{ sec}$$

$$d_{\text{total}} = 288 + 6300 = 6588 \text{ sec}$$

Final Answer

$d_{\text{total}} = 6588$ s $= 109.8$ minutes ≈ 1 hr 49 min 48 s. Fewer cars means less time spent queuing at each tollbooth, so the total delay drops slightly.

Note: If your version of the textbook specifies a different per-car service time or a different tollbooth spacing, substitute those values into $d_{\text{total}} = 3N(\text{service time}) + 2 \cdot \dfrac{\text{segment length}}{100 \text{ km/hr}}$ – the method is unaffected.

Problem 6 Propagation Delay and Transmission Delay Fundamentals

Problem Statement

Hosts A and B are connected by a single link of rate R bps, separated by m meters, with propagation speed s m/sec. Host A sends a packet of L bits. **(a)**–**(f)** Express delays and locate bits at specific times (see full statement). **(g)** With $s = 2.5 \times 10^8$ m/s, $L = 1500$ bytes, $R = 10$ Mbps, find m so that $d_{\text{prop}} = d_{\text{trans}}$.

Solution

(a) Propagation delay is distance over speed:

Final Answer

$$d_{\text{prop}} = \frac{m}{s}$$

(b) Transmission delay is the time to push all L bits onto the link at rate R:

Final Answer

$$d_{\text{trans}} = \frac{L}{R}$$

(c) Ignoring processing and queuing delay, the end-to-end delay is the sum:

Final Answer

$$d_{\text{end-to-end}} = d_{\text{prop}} + d_{\text{trans}} = \frac{m}{s} + \frac{L}{R}$$

(d) At $t = d_{\text{trans}}$, Host A has just finished pushing the last bit onto the link.

Final Answer

The last bit is just leaving Host A (entering the link at the sending end).

(e) If $d_{\text{prop}} > d_{\text{trans}}$, then at $t = d_{\text{trans}}$ the first bit has been propagating for only d_{trans} seconds and has covered a distance $s \cdot d_{\text{trans}} < s \cdot d_{\text{prop}} = m$.

Final Answer

The first bit is still *in the link*, partway between A and B (it has not yet reached B).

(f) If $d_{\text{prop}} < d_{\text{trans}}$, the first bit only needed $d_{\text{prop}} < d_{\text{trans}}$ seconds to reach B, so by $t = d_{\text{trans}}$ it arrived earlier.

Final Answer

The first bit has already arrived at Host B.

(g) Set $d_{\text{prop}} = d_{\text{trans}}$, i.e., $\dfrac{m}{s} = \dfrac{L}{R} \Rightarrow m = \dfrac{sL}{R}$.

With $L = 1500$ bytes $= 12{,}000$ bits, $R = 10 \times 10^6$ bps:

$$d_{\text{trans}} = \frac{12{,}000}{10 \times 10^6} = 1.2 \times 10^{-3} \text{ s} = 1.2 \text{ ms}$$

$$m = s \cdot d_{\text{trans}} = (2.5 \times 10^8)(1.2 \times 10^{-3}) = 3 \times 10^5 \text{ m}$$

Final Answer

$m = 300{,}000$ m $= 300$ km

Problem 7 VoIP End-to-End Delay

Problem Statement

Host A digitizes voice at 64 kbps and groups bits into 56-byte packets, sent over a single link (10 Mbps, 10 ms propagation) to Host B, which decodes each packet as it fully arrives. How much time elapses from when a bit is created (at A) to when it is decoded (at B)?

Solution

Three delay components apply to a bit, using the worst case of a bit that is the *first* bit placed into a packet (it must wait for the rest of the packet to fill before the packet can be sent):

1. Packetization delay. A packet holds $L = 56$ bytes $= 448$ bits. At a generation rate of 64,000 bits/sec, the time to accumulate a full packet is:

$$d_{\text{packetize}} = \frac{448 \text{ bits}}{64{,}000 \text{ bits/sec}} = 7 \times 10^{-3} \text{ s} = 7 \text{ ms}$$

2. Transmission delay onto the 10 Mbps link:

$$d_{\text{trans}} = \frac{448}{10 \times 10^6} = 4.48 \times 10^{-5} \text{ s} = 0.0448 \text{ ms}$$

3. Propagation delay (given):

$$d_{\text{prop}} = 10 \text{ ms}$$

Total elapsed time (bit creation → full packet assembled → transmitted → propagated → decoded):

$$d_{\text{total}} = 7 + 0.0448 + 10 = 17.0448 \text{ ms}$$

Final Answer

$d_{\text{total}} \approx 17.04$ ms

Problem 8 Circuit Switching vs. Packet Switching: Shared 10 Mbps Link

Problem Statement

Users share a 10 Mbps link. Each user needs 200 kbps when active, but is active only 10% of the time. **(a)** Under circuit switching, how many users can be supported? **(b)** Under packet switching, find the probability a given user is transmitting. **(c)** With 120 users, find the probability that exactly n are transmitting simultaneously. **(d)** Find the probability that 51 or more users are transmitting simultaneously.

Solution

(a) Circuit switching dedicates a fixed 200 kbps slice to each user regardless of activity:

Final Answer

$$\frac{10 \text{ Mbps}}{200 \text{ kbps}} = \mathbf{50} \text{ users}$$

(b) We are told directly that each user transmits 10% of the time:

Final Answer

$$p = 0.1$$

(c) With 120 independent users each active with probability $p = 0.1$, the number transmitting simultaneously, X, is Binomial$(120, 0.1)$:

Final Answer

$$P(X = n) = \binom{120}{n}(0.1)^n(0.9)^{120-n}, \qquad n = 0, 1, \ldots, 120$$

(d) The link can serve at most 10 Mbps/200 kbps = 50 simultaneous active users, so we want $P(X \geq 51)$:

$$P(X \geq 51) = \sum_{n=51}^{120} \binom{120}{n}(0.1)^n(0.9)^{120-n}$$

The mean of X is $120(0.1) = 12$ and its standard deviation is $\sqrt{120(0.1)(0.9)} \approx 3.29$, so 51 users is roughly 12 standard deviations above the mean – an extreme tail event. Evaluating the sum numerically:

Final Answer

$$P(X \geq 51) \approx 2.06 \times 10^{-20}$$

This is astronomically small – for all practical purposes it never happens. This is exactly the point of the example: packet switching's statistical multiplexing lets 120 bursty users share a link sized for only 50 simultaneous active users, at essentially zero risk of overload, whereas circuit switching would flatly refuse the 51st user.

Problem 9 Circuit Switching on a 1 Gbps Link

Problem Statement

Repeat the packet-vs-circuit-switching comparison of Section 1.3, but with the 1 Mbps link replaced by a 1 Gbps link. Users generate 100 kbps when busy, and are busy with probability $p = 0.1$. **(a)** What is N, the max number of users supported simultaneously under circuit switching? **(b)** For M users under packet switching, give a formula for the probability that more than N users are sending data at once.

Solution

(a) Each user needs a dedicated 100 kbps circuit:

Final Answer

$$N = \frac{1 \text{ Gbps}}{100 \text{ kbps}} = \frac{10^9}{10^5} = \mathbf{10{,}000} \text{ users}$$

(b) With M users, each independently active with probability p, the number simultaneously active $X \sim \text{Binomial}(M, p)$. The probability that more than N users are sending at once is:

Final Answer

$$P(X > N) = \sum_{k=N+1}^{M} \binom{M}{k} p^k (1-p)^{M-k}$$

Problem 10 End-to-End Delay Through Two Packet Switches

Problem Statement

A packet of length L travels from A to a destination over three links (rates R_1, R_2, R_3; lengths/propagation speeds d_i, s_i) connected by two packet switches, each adding a processing delay d_{proc}. **(a)** Give a general expression (no queuing delay). **(b)** Compute the numeric delay for $L = 1500$ bytes, $s_i = 2.5 \times 10^8$ m/s (all links), $R_i = 2.5$ Mbps (all links), $d_{\text{proc}} = 3$ ms/switch, $d_1 = 5000$ km, $d_2 = 4000$ km, $d_3 = 1000$ km.

Solution

(a) General expression. Each of the 3 links contributes a transmission delay L/R_i and a propagation delay d_i/s_i; each of the 2 switches contributes a processing delay d_{proc} (store-and-forward, so no other overlap):

Final Answer

$$d_{\text{end-to-end}} = \sum_{i=1}^{3} \left(\frac{L}{R_i} + \frac{d_i}{s_i} \right) + 2\, d_{\text{proc}}$$

(b) Numeric evaluation. $L = 1500$ bytes $= 12{,}000$ bits.

Transmission delay (same $R = 2.5 \times 10^6$ bps on all 3 links):

$$3 \times \frac{12{,}000}{2.5 \times 10^6} = 3 \times 4.8 \text{ ms} = 14.4 \text{ ms}$$

Propagation delay (same $s = 2.5 \times 10^8$ m/s on all 3 links):

$$\frac{5 \times 10^6}{2.5 \times 10^8} + \frac{4 \times 10^6}{2.5 \times 10^8} + \frac{1 \times 10^6}{2.5 \times 10^8} = 20 + 16 + 4 = 40 \text{ ms}$$

Processing delay: 2×3 ms $= 6$ ms.

Final Answer

$$d_{\text{end-to-end}} = 14.4 + 40 + 6 = \mathbf{60.4} \text{ ms}$$

Problem 11 End-to-End Delay With Cut-Through Switching

Problem Statement

Repeat Problem P10 with $R_1 = R_2 = R_3 = R$ and $d_{\text{proc}} = 0$, but now the packet switches use *cut-through* forwarding: each switch retransmits every bit as soon as it arrives, rather than waiting for the whole packet (no store-and-forward).

Solution

With cut-through switching the packet is effectively pipelined through all three links simultaneously: the transmission delay of L/R is incurred only *once* (by the source, to push the packet onto the first link), because each switch begins relaying each bit almost immediately rather than buffering the entire packet. After that, the (already-in-flight) bits simply need to propagate across the physical distance of all three links, one after another, at speed s.

$$d_{\text{end-to-end}} = \frac{L}{R} + \frac{d_1 + d_2 + d_3}{s}$$

Using $L = 12{,}000$ bits, $R = 2.5 \times 10^6$ bps, $s = 2.5 \times 10^8$ m/s, and $d_1 + d_2 + d_3 = 10^7$ m:

$$\frac{L}{R} = 4.8 \text{ ms}, \qquad \frac{d_1 + d_2 + d_3}{s} = \frac{10^7}{2.5 \times 10^8} = 40 \text{ ms}$$

Final Answer

$$d_{\text{end-to-end}} = 4.8 + 40 = \mathbf{44.8} \text{ ms}$$

This is $60.4 - 44.8 = 15.6$ ms less than the store-and-forward case in P10 – exactly the two extra L/R transmission delays ($2 \times 4.8 = 9.6$ ms) plus the two d_{proc} processing delays ($2 \times 3 = 6$ ms) that cut-through switching eliminates.

Problem 12 Queuing Delay Behind Packets Already in the Buffer

Problem Statement

A packet arrives at a switch's outbound link. One other packet is halfway done being transmitted, and four more packets are already queued ahead of the new arrival. All packets are 1500 bytes; the link rate is 2.5 Mbps. Find the queuing delay for the new packet. Generalize to packet length L, rate R, x bits already transmitted of the current packet, and n packets already queued.

Solution

The new packet must wait for: (1) the *remainder* of the packet currently mid-transmission, and (2) the full transmission of each of the n packets ahead of it in the queue.

General formula: if x bits of the current packet (length L) have already been sent, the remaining transmission time for it is $(L - x)/R$. Each of the n queued packets needs L/R.

Final Answer

$$d_{\text{queue}} = \frac{L - x}{R} + n \cdot \frac{L}{R} = \frac{(L - x) + nL}{R}$$

Numeric case: "halfway done" means $x = L/2$, so the remaining time for that packet is $\dfrac{L/2}{R} = \dfrac{1}{2} \cdot \dfrac{L}{R}$, and there are $n = 4$ full packets queued ahead:

$$d_{\text{queue}} = \frac{1}{2} \cdot \frac{L}{R} + 4 \cdot \frac{L}{R} = 4.5 \cdot \frac{L}{R}$$

With $L = 1500$ bytes $= 12{,}000$ bits and $R = 2.5 \times 10^6$ bps: $L/R = 4.8$ ms.

Final Answer

$$d_{\text{queue}} = 4.5 \times 4.8 \text{ ms} = \mathbf{21.6} \text{ ms}$$

Problem 13 Average Queuing Delay for a Burst of N Packets

Problem Statement

(a) N packets of length L arrive simultaneously to an idle link of rate R. Find the average queuing delay for the N packets. **(b)** Now suppose such a batch of N packets arrives every LN/R seconds. What is the average queuing delay of a packet?

Solution

(a) Number the packets $1, 2, \ldots, N$ in transmission order. Packet 1 is transmitted immediately (queuing delay 0). Packet 2 must wait for packet 1 to finish transmitting (L/R). In general, packet k waits for the $k-1$ packets ahead of it: queuing delay $= (k-1)\dfrac{L}{R}$.

$$\overline{d_{\text{queue}}} = \frac{1}{N}\sum_{k=1}^{N}(k-1)\frac{L}{R} = \frac{L}{RN}\sum_{k=0}^{N-1} k = \frac{L}{RN}\cdot\frac{N(N-1)}{2}$$

Final Answer

$$\overline{d_{\text{queue}}} = \frac{(N-1)}{2}\cdot\frac{L}{R}$$

(b) The interarrival period between batches is exactly LN/R – precisely the time required to transmit an entire batch of N packets. This means the queue is completely drained (empty) the instant the next batch arrives; no leftover backlog ever carries over from one batch to the next. Each batch therefore experiences exactly the same queuing pattern computed in part (a).

Final Answer

$$\overline{d_{\text{queue}}} = \frac{(N-1)}{2}\cdot\frac{L}{R} \quad \text{(unchanged from part (a))}$$

Problem 14 Total Delay as a Function of Traffic Intensity

Problem Statement

Let $I = La/R$ be the traffic intensity. Suppose the queuing delay takes the form $\dfrac{IL/R}{1-I}$ for $I < 1$. **(a)** Give a formula for the total delay (queuing + transmission). **(b)** Plot the total delay as a function of L/R.

Solution

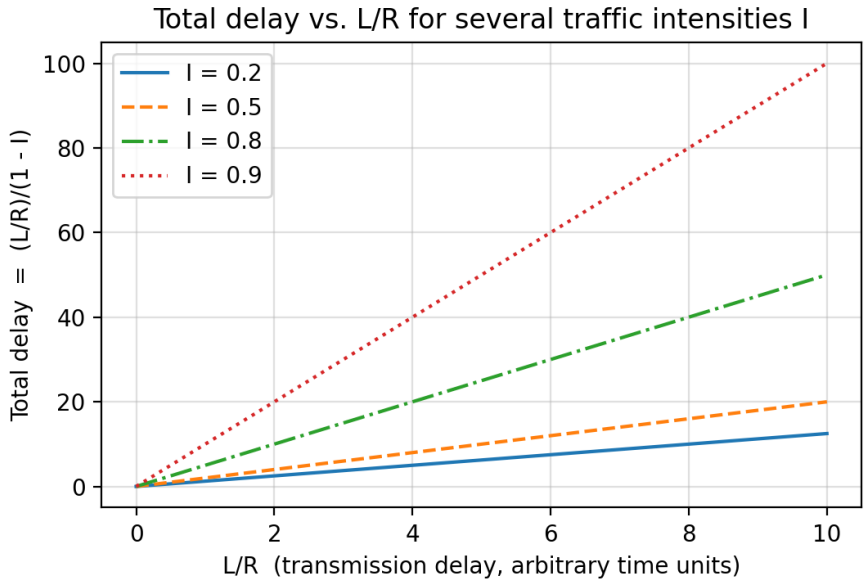
(a) Total delay is transmission delay plus queuing delay:

$$d_{\text{total}} = \frac{L}{R} + \frac{I}{1-I} \cdot \frac{L}{R} = \frac{L}{R}\left(1 + \frac{I}{1-I}\right) = \frac{L}{R} \cdot \frac{(1-I)+I}{1-I}$$

Final Answer

$$d_{\text{total}} = \frac{L/R}{1-I}$$

(b) For a fixed traffic intensity I, d_{total} is a *straight line through the origin* in L/R, with slope $1/(1-I)$. As $I \to 1^-$ the slope blows up, illustrating the well-known result that delay grows without bound as the link approaches saturation. The figure below plots d_{total} vs. L/R for several values of I:



Final Answer

$d_{\text{total}} = \dfrac{L/R}{1-I}$ – linear in L/R, with the slope increasing sharply as I approaches 1 (heavier traffic intensity makes the link far more delay-sensitive to any increase in packet transmission time).

Problem 15 Total Delay in Terms of Arrival and Service Rates

Problem Statement

Let a be the packet arrival rate (packets/sec) and μ the link's transmission (service) rate (packets/sec). Using the total-delay formula from the previous problem, derive a formula for total delay in terms of a and μ.

Solution

The time to transmit one packet, L/R, is exactly the reciprocal of the service rate: $L/R = 1/\mu$. Similarly, the traffic intensity $I = La/R$ is exactly $I = a/\mu$ (arrival rate over service rate) when both are expressed in packets/sec. Substituting into $d_{\text{total}} = \dfrac{L/R}{1-I}$:

$$d_{\text{total}} = \frac{1/\mu}{1 - a/\mu} = \frac{1/\mu}{(\mu - a)/\mu} = \frac{1}{\mu - a}$$

Final Answer

$$d_{\text{total}} = \frac{1}{\mu - a}$$

This is the classic M/M/1 queueing-theory result: total delay diverges as the arrival rate a approaches the service rate μ.

Problem 16 Little's Formula

Problem Statement

Let N be the average number of packets in the buffer plus the packet being transmitted, a the arrival rate, and d the average total delay. Little's formula: $N = a \cdot d$. Given an average buffer occupancy of 100 packets, average queuing delay 20 ms, and link transmission rate 100 packets/sec, find the average packet arrival rate (assuming no loss).

Solution

The total delay d used in Little's formula must include *both* the queuing delay *and* the transmission (service) delay, since N counts packets in the buffer *plus* the one currently being transmitted.

$$\text{Transmission delay per packet} = \frac{1}{\text{transmission rate}} = \frac{1}{100 \text{ packets/sec}} = 10 \text{ ms}.$$

$$d = d_{\text{queue}} + d_{\text{trans}} = 20 \text{ ms} + 10 \text{ ms} = 30 \text{ ms} = 0.03 \text{ s}$$

Applying Little's formula, $N = a \cdot d \;\Rightarrow\; a = N/d$:

$$a = \frac{100}{0.03}$$

Final Answer

$$a \approx \mathbf{3333.3} \text{ packets/sec}$$

Problem 17 Generalizing Equation 1.2 for Heterogeneous Nodes

Problem Statement

(a) Generalize Equation 1.2 (Section 1.4.3) for heterogeneous processing rates, transmission rates, and propagation delays. **(b)** Repeat (a), now also including an average queuing delay d_{queue} at each node.

Solution

Equation 1.2 in the text gives the end-to-end delay assuming *identical* processing, transmission, and propagation delay at every one of N links/nodes: $d_{\text{end-to-end}} = N(d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}})$. When these differ at every node $i = 1, \ldots, N$, we simply sum the per-node contributions rather than multiply a single value by N.

(a)

Final Answer

$$d_{\text{end-to-end}} = \sum_{i=1}^{N} (d_{\text{proc},i} + d_{\text{trans},i} + d_{\text{prop},i}) = \sum_{i=1}^{N} \left(d_{\text{proc},i} + \frac{L}{R_i} + d_{\text{prop},i} \right)$$

(b) Adding a per-node average queuing delay $d_{\text{queue},i}$:

Final Answer

$$d_{\text{end-to-end}} = \sum_{i=1}^{N} (d_{\text{proc},i} + d_{\text{trans},i} + d_{\text{prop},i} + d_{\text{queue},i})$$

Problem 18 Hands-On Traceroute Exercise

Problem Statement

Perform Traceroute between a source and destination on the same continent, at three different hours of the day. **(a)** Find the average and standard deviation of round-trip delay at each hour. **(b)** Find the number of routers in the path at each hour; did the path change? **(c)** Identify the number of ISP networks traversed; do the largest delays occur at ISP peering points? **(d)** Repeat for a source and destination on different continents, and compare intra- vs. inter-continent results.

Solution

This problem is a hands-on *empirical measurement exercise* rather than something with a fixed numeric answer – it asks you to run `traceroute` (macOS/Linux) or `tracert` (Windows) from your own machine, at your own physical location, at three different times of day. I don't have network access in this environment, and even if I did, running it from here would measure the path from *my* network location rather than yours, which would defeat the purpose of the exercise (the whole point is to see your own local ISP's routing and time-of-day congestion patterns). Here's exactly how to do it and what each part is asking you to compute:

1. **Run the command.** Pick a destination on your own continent (e.g., a well-known site's server) and run, e.g. on macOS/Linux: `traceroute example.com`, or on Windows: `tracert example.com`. Do this at three noticeably different hours (e.g., early morning, midday, late evening) to capture different network load conditions.
2. **Part (a).** Each hop line in the output reports (typically) three round-trip time samples. Take the RTT reported for the *final* hop (the destination) across your three runs at that hour, or average all per-hop samples for that run, and compute the sample mean $\bar{x} = \frac{1}{n}\sum x_i$ and sample standard deviation $s = \sqrt{\frac{1}{n-1}\sum(x_i - \bar{x})^2}$.
3. **Part (b).** Count the number of hop lines (routers) printed before reaching the destination, for each of the three runs. Compare the router IP addresses/hostnames hop-by-hop across the three runs – if they differ, the path changed (which can happen due to routing changes or load balancing).
4. **Part (c).** Group consecutive hops by matching hostname patterns or IP address blocks (organizations often name their routers with a recognizable substring, e.g., `...comcast.net`, `...level3.net`) to estimate how many distinct ISPs the path crosses. Compare the per-hop delay *increments* (the jump in RTT from one hop to the next) – delay jumps often occur right at the hop where the path crosses from one ISP's network into another's (a "peering" link), because peering links are sometimes less provisioned than each ISP's own backbone.
5. **Part (d).** Repeat the exact same measurements to a destination on a different continent, and compare: intercontinental paths are almost always longer in hop count and substantially higher in average RTT (dominated by the physical propagation delay across a much greater distance, plus more inter-ISP hops), and can show larger RTT variance due to crossing more networks and possibly undersea cable links.

(If you run this and want help interpreting your actual output, paste the traceroute results and I'll walk through the analysis with you.)

Problem 19 Metcalfe's Law

Problem Statement

Metcalfe's law states that the value of a network is proportional to the square of the number of connected users. If each of n users sends one message to each of the other users, how many messages are sent? Does this support Metcalfe's law?

Solution

Each of the n users sends exactly one message to each of the other $n-1$ users, so the total number of messages is:

Final Answer

$$\text{Total messages} = n(n-1)$$

Expanding, $n(n-1) = n^2 - n$, which for large n is dominated by the n^2 term ($n^2 - n \approx n^2$). Since the total number of pairwise messages grows quadratically in n – proportional to n^2 – this result **does support Metcalfe's law**: a network's aggregate connectivity/communication capacity (a natural proxy for its "value") scales with the *square* of the number of users, not linearly, because every new user adds a potential communication link to every existing user.

Problem 20 Throughput With M Client-Server Pairs

Problem Statement

Extend the throughput example of Figure 1.21(b) from 10 client-server pairs to M pairs, where R_s, R_c, and R denote the server-link, client-link, and shared network-link rates. Derive a general expression for throughput.

Solution

Each client-server pair has its own dedicated access links (R_s into the network from its server, R_c out to its client), but all M pairs share the single core network link of rate R. Two effects can bottleneck any one flow's throughput:

1. its own access links, R_s and R_c, which no other flow contends for, and
2. its fair share of the shared core link, which (assuming the M flows split R evenly) is R/M.

The achievable throughput per pair is limited by whichever of these three quantities is smallest:

Final Answer

$$\text{Throughput per pair} = \min\left(R_s,\ R_c,\ \frac{R}{M}\right)$$

This matches the textbook's 10-pair example exactly when $M = 10$: $\min(R_s, R_c, R/10)$.

Problem 21 Maximum Throughput Over M Parallel Paths

Problem Statement

Referring to Figure 1.20(b), suppose there are M link-disjoint paths between server and client. Path k consists of N links with rates $R_1^k, R_2^k, \ldots, R_N^k$. **If** the server can use only one path, what is the max throughput? **If** it can use all M paths simultaneously, what is the max throughput?

Solution

Bottleneck of a single path. The throughput achievable along any one path k is limited by the slowest (bottleneck) link on that path:

$$\text{Throughput of path } k = \min_{1 \le j \le N} R_j^k$$

Single-path case. If the server must pick just one path to use, it will naturally choose whichever path has the *largest* bottleneck rate, to maximize throughput:

Final Answer

$$\text{Max throughput (single path)} = \max_{1 \le k \le M} \left(\min_{1 \le j \le N} R_j^k \right)$$

All-paths case. Because no two paths share any link, the paths are completely independent and can carry traffic in parallel without interfering with one another. The maximum aggregate throughput is simply the sum of each path's individual bottleneck rate:

Final Answer

$$\text{Max throughput (all } M \text{ paths)} = \sum_{k=1}^{M} \left(\min_{1 \le j \le N} R_j^k \right)$$

Problem 22 Probability of Successful Delivery and Expected Retransmissions

Problem Statement

Referring to Figure 1.20(b), each link between server and client has an independent packet-loss probability p. What is the probability a packet is successfully received? On average, how many times will the server retransmit for the client to successfully receive it?

Solution

Assumption: Figure 1.20's server-to-client path (as also referenced by the first/second-link description in Problem P23) consists of exactly **two** links in series, each with independent loss probability p.



Probability of success. A packet survives only if it is *not* lost on link 1 *and not* lost on link 2. Since losses are independent:

Final Answer

$$P(\text{success}) = (1-p)(1-p) = (1-p)^2$$

Expected number of retransmissions. Each end-to-end send attempt independently succeeds with probability $P_s = (1-p)^2$. The number of attempts needed until the first success follows a Geometric(P_s) distribution, with mean number of *attempts* $= 1/P_s$. The number of *re*transmissions is one less than the number of attempts (the first attempt isn't a "re"transmission):

Final Answer

$$E[\text{retransmissions}] = \frac{1}{(1-p)^2} - 1 = \frac{1-(1-p)^2}{(1-p)^2}$$

(If your edition's Figure 1.20(b) shows a different number of links k in the path, the same reasoning gives $P(success) = (1-p)^k$ and $E[retransmissions] = (1-p)^{-k} - 1$.)

Problem 23 Packet-Pair Bottleneck-Bandwidth Probing

Problem Statement

Figure 1.20(a): the bottleneck along the server-to-client path is the first link, rate R_s. Two packets of size L bits are sent back-to-back; both links have propagation delay d_{prop}. **(a)** What is the packet inter-arrival time at the destination? **(b)** Now suppose the second link is the bottleneck ($R_c < R_s$). Can the second packet queue at the router's input to link 2? If the server sends packet 2 exactly T seconds after packet 1, how large must T be to avoid any such queuing?

Solution



(a) $R_s < R_c$ (link 1 is the bottleneck). Track the *last bit* of each packet.

- Packet 1 finishes transmission at the server at $t = L/R_s$, arrives at the router at $t = L/R_s + d_{\text{prop}}$, is immediately forwarded (store-and-forward) and finishes transmitting onto link 2 at $t = L/R_s + d_{\text{prop}} + L/R_c$, and arrives at the client at $t_1 = L/R_s + L/R_c + 2d_{\text{prop}}$.
- Packet 2 begins transmission the instant packet 1 finishes on link 1 (back-to-back), so it finishes transmission at the server at $t = 2L/R_s$, arrives at the router at $t = 2L/R_s + d_{\text{prop}}$.

Because link 1 is the bottleneck ($R_s < R_c$), the router *always* finishes relaying packet 1 onto the (faster) link 2 before packet 2 even arrives at the router, so packet 2 experiences no queuing at the router. It finishes transmitting onto link 2 at $t = 2L/R_s + d_{\text{prop}} + L/R_c$ and arrives at the client at $t_2 = 2L/R_s + L/R_c + 2d_{\text{prop}}$.

Final Answer

$$\text{Inter-arrival time} = t_2 - t_1 = \frac{L}{R_s}$$

The gap at the receiver equals exactly the transmission time on the *bottleneck* link – this is the basis of "packet-pair" bandwidth-probing techniques used to measure a path's bottleneck rate.

(b) $R_c < R_s$ (link 2 is now the bottleneck). Yes, queuing at the router is possible now. Packet 1 arrives at the router at $t = L/R_s + d_{\text{prop}}$ and, since link 2 is now slow, doesn't finish being forwarded onto link 2 until $t = L/R_s + d_{\text{prop}} + L/R_c$. If packet 2 is sent T seconds after packet 1 (i.e., leaves the server at $t = T$), it arrives at the router at $t = T + L/R_s + d_{\text{prop}}$. If this arrival happens *before* the router has finished forwarding packet 1 (i.e., before $t = L/R_s + d_{\text{prop}} + L/R_c$), packet 2 must sit in the router's queue waiting for link 2 to free up.

To guarantee *no* queuing, packet 2 must arrive at the router no earlier than the moment the router finishes with packet 1:

$$T + \frac{L}{R_s} + d_{\text{prop}} \;\geq\; \frac{L}{R_s} + d_{\text{prop}} + \frac{L}{R_c}$$

Final Answer

$$T \geq \frac{L}{R_c}$$

That is, the server must space its packets apart by at least the transmission time on the (now-bottleneck) second link to guarantee the router's queue for link 2 never builds up.

Problem 24 50 TB Boston-to-LA: Dedicated Link vs. FedEx

Problem Statement

You need to urgently move 50 terabytes of data from Boston to Los Angeles. You have a dedicated 100 Mbps link. Would you use the link or FedEx overnight delivery? Explain.

Solution

Convert 50 TB to bits: 50×10^{12} bytes $\times\, 8 = 4 \times 10^{14}$ bits.

Time over the 100 Mbps link:

$$t = \frac{4 \times 10^{14} \text{ bits}}{100 \times 10^{6} \text{ bps}} = 4 \times 10^{6} \text{ s} \approx 46.3 \text{ days}$$

Time via FedEx overnight: roughly 1 day, since 50 TB fits comfortably on a small number of commodity hard drives that can be physically shipped.

Final Answer

Use FedEx. ≈1 day (FedEx) vs. ≈46.3 days (100 Mbps link) – FedEx is roughly 46× faster. This is the classic "never underestimate the bandwidth of a station wagon full of storage media" (a.k.a. "sneakernet") result: for extremely large data volumes, the effective throughput of physically shipping storage media can dramatically exceed even a solid dedicated network link, because a truck/plane full of drives carries an enormous number of bits per unit time even though its per-bit *latency* is a full day.

Problem 25 Bandwidth-Delay Product (R = 5 Mbps)

Problem Statement

Hosts A and B are 20,000 km apart, linked directly at $R = 5$ Mbps, with propagation speed 2.5×10^8 m/s. **(a)** Calculate the bandwidth-delay product. **(b)** For an 800,000-bit file sent continuously, what is the max number of bits in the link at any time? **(c)** Interpret the bandwidth-delay product. **(d)** What is the width (meters) of a bit? Is it longer than a football field? **(e)** Give a general expression for the width of a bit.

Solution

$m = 20{,}000$ km $= 2 \times 10^7$ m, $s = 2.5 \times 10^8$ m/s, $R = 5 \times 10^6$ bps.

(a)

$$d_{\text{prop}} = \frac{m}{s} = \frac{2 \times 10^7}{2.5 \times 10^8} = 0.08 \text{ s}$$

Final Answer

$$R \cdot d_{\text{prop}} = (5 \times 10^6)(0.08) = \mathbf{400{,}000} \text{ bits}$$

(b) The file (800,000 bits) is *larger* than the bandwidth-delay product (400,000 bits), so once the link is "full," it can never hold more than $R \cdot d_{\text{prop}}$ bits in flight simultaneously – bits continue to drain out the far end at rate R exactly as fast as new ones enter.

Final Answer

Max bits in the link at once = **400,000** bits (the bandwidth-delay product itself, since the file is bigger than the pipe can hold).

(c)

Final Answer

The bandwidth-delay product represents the maximum number of bits that can be "in flight" (in transit, not yet acknowledged/received) in the link at any instant – i.e., the *volume* of the link viewed as a pipe, where bandwidth R is analogous to the pipe's cross-sectional area and propagation delay d_{prop} is analogous to its length.

(d) The physical length occupied by one bit equals the distance light (well, the signal) travels during the time it takes to transmit one bit, $1/R$ seconds:

$$\text{width} = s \cdot \frac{1}{R} = \frac{2.5 \times 10^8}{5 \times 10^6} = 50 \text{ m}$$

A football field is 100 yards ≈ 91.4 m (not counting end zones).

Final Answer

Width of a bit = **50** meters – *shorter* than a football field (about half its length).

(e)

Final Answer

$$\text{width of a bit} = \frac{s}{R}$$

(Note this doesn't actually depend on the link length m – a bit's physical "width" is purely a function of propagation speed and transmission rate; m only determines *how many* bit-widths fit along the whole link.)

Problem 26 Choosing R So a Bit's Width Equals the Link Length

Problem Statement

Referring to Problem P25, for what value of R is the width of a bit as long as the length of the link?

Solution

We need $\frac{s}{R} = m$, i.e., $R = \frac{s}{m}$. Using $s = 2.5 \times 10^8$ m/s and $m = 2 \times 10^7$ m:

$$R = \frac{2.5 \times 10^8}{2 \times 10^7}$$

Final Answer

$$R = 12.5 \text{ bps}$$

At this (very low) transmission rate, a single bit's transmission takes so long that its leading edge doesn't even reach Host B before the entire bit has finished being pushed onto the link – the whole 20,000 km link holds exactly one bit's worth of signal at a time.

Problem 27 Problem P25 Repeated with R = 500 Mbps

Problem Statement

Consider Problem P25 again, now with $R = 500$ Mbps. **(a)** Calculate the bandwidth-delay product. **(b)** For the 800,000-bit file sent continuously, what is the max number of bits in the link at any time? **(c)** What is the width (meters) of a bit?

Solution

$d_{\text{prop}} = 0.08$ s (unchanged – it doesn't depend on R). $R = 500 \times 10^6$ bps.

(a)

Final Answer

$$R \cdot d_{\text{prop}} = (500 \times 10^6)(0.08) = \mathbf{4 \times 10^7} \text{ bits} = \mathbf{40{,}000{,}000} \text{ bits}$$

(b) Now the bandwidth-delay product (40,000,000 bits) is far *larger* than the 800,000-bit file. In fact, the transmission delay of the whole file, $800{,}000/(500 \times 10^6) = 1.6$ ms, is much shorter than the 80 ms propagation delay – so the *entire* file finishes leaving the source and is "in flight," spread out along the link, well before the first bit even reaches Host B.

Final Answer

Max bits in the link at once = **800,000** bits (the whole file fits inside the pipe simultaneously, since the file is much smaller than the bandwidth-delay product).

(c)

$$\text{width} = \frac{s}{R} = \frac{2.5 \times 10^8}{500 \times 10^6}$$

Final Answer

$$\text{width of a bit} = \mathbf{0.5} \text{ meters}$$

(100× smaller than in P25(d), because R increased 100×.)

Problem 28 Continuous vs. Stop-and-Wait Transfer Time

Problem Statement

Referring again to Problem P25 ($R = 5$ Mbps, $d_{\text{prop}} = 0.08$ s): **(a)** How long does it take to send the 800,000-bit file continuously? **(b)** Now the file is split into 20 packets of 40,000 bits each; each packet must be acknowledged (negligible ack transmission time) before the next is sent. How long does it take to send the whole file? **(c)** Compare (a) and (b).

Solution

(a) Continuous transmission.

$$d_{\text{a}} = \frac{L}{R} + d_{\text{prop}} = \frac{800{,}000}{5 \times 10^6} + 0.08 = 0.16 + 0.08$$

Final Answer

$$d_{\text{a}} = \mathbf{0.24}\ \text{s}$$

(b) Stop-and-wait, 20 packets of $L_{\text{pkt}} = 40{,}000$ bits each. Each packet (except the last) requires a full round trip before the next can be sent: transmit the packet (L_{pkt}/R), it propagates to the client (d_{prop}), the (negligible-size) ack propagates back (d_{prop}):

$$\text{per-packet cycle} = \frac{L_{\text{pkt}}}{R} + 2d_{\text{prop}} = \frac{40{,}000}{5 \times 10^6} + 2(0.08) = 0.008 + 0.16 = 0.168\ \text{s}$$

Packets 1 through 19 each require this full 0.168 s cycle before the next packet can begin. The 20th packet only needs to be transmitted and propagate to the client (its ack isn't needed once the file has fully arrived):

$$d_{\text{b}} = 19 \times 0.168 + \left(\frac{L_{\text{pkt}}}{R} + d_{\text{prop}}\right) = 19(0.168) + (0.008 + 0.08)$$

$$d_{\text{b}} = 3.192 + 0.088$$

Final Answer

$$d_{\text{b}} = \mathbf{3.28}\ \text{s}$$

(c) Comparison.

Final Answer

Stop-and-wait (3.28 s) takes about $3.28/0.24 \approx 13.7\times$ longer than sending the file continuously as one message (0.24 s). Waiting for an acknowledgment (a full $2d_{\text{prop}}$ round trip) after every small packet is extremely costly whenever propagation delay is large relative to the transmission delay of each packet – exactly the "long, fat network" scenario this problem illustrates, and precisely the motivation for pipelined/windowed protocols (like TCP's sliding window) instead of simple stop-and-wait.

Problem 29 Geostationary Satellite Microwave Link

Problem Statement

A 10 Mbps microwave link connects a geostationary satellite and its base station (propagation speed 2.4×10^8 m/s). Every minute the satellite takes a digital photo and sends it down. **(a)** What is the propagation delay of the link? **(b)** What is the bandwidth-delay product? **(c)** Let x be the photo size. What is the minimum x for the link to be continuously transmitting?

Solution

A geostationary orbit's altitude is approximately 36,000 km $= 3.6 \times 10^7$ m above the Earth's surface, which is the standard one-way link distance used for this classic problem.

(a)

$$d_{\text{prop}} = \frac{3.6 \times 10^7 \text{ m}}{2.4 \times 10^8 \text{ m/s}}$$

Final Answer

$$d_{\text{prop}} = 0.15 \text{ s} = 150 \text{ ms}$$

(b)

$$R \cdot d_{\text{prop}} = (10 \times 10^6)(0.15)$$

Final Answer

$$R \cdot d_{\text{prop}} = 1{,}500{,}000 \text{ bits}$$

(c) For the link to never sit idle, each photo must take *at least* as long to transmit as the 60-second interval between photos being generated – otherwise the link finishes sending one photo and has to wait for the next to be captured:

$$\frac{x}{R} \geq 60 \text{ s} \implies x \geq R \times 60 = (10 \times 10^6)(60)$$

Final Answer

$$x_{\min} = 6 \times 10^8 \text{ bits} = 600 \text{ Mbits} = 75 \text{ MB}$$

Any photo smaller than this leaves the link idle for part of each 60-second cycle; a photo of exactly this size (or larger, if the satellite buffers backlog) keeps the link continuously busy.

Problem 30 Layering: Headers in the Airline Analogy

Problem Statement

Consider the airline-travel analogy for protocol layering, and the addition of headers to protocol data units as they move down the stack. Is there an equivalent notion of "header" information added to passengers and baggage as they move down the airline's protocol stack?

Solution

Final Answer

Yes. Just as each network protocol layer prepends a header carrying control information for its peer layer at the other end, the airline system attaches analogous "header" information to both passengers and baggage at check-in:

- **Baggage claim tags**, attached to each suitcase at check-in, encode routing information – destination airport code, flight number, any connecting-flight legs. This is exactly analogous to a network/link-layer header: it lets intermediate handling systems (baggage carts, sorting facilities, transfer belts at a layover airport) route the "payload" (the suitcase) correctly *without ever needing to know what's inside it* – just as a router forwards a packet based on its header without inspecting the application data.
- **Boarding passes**, issued to each passenger, similarly encode routing/identification information: flight number, seat assignment, gate. Gate agents and flight crew use this "header" to route and seat the "payload" (the passenger) correctly.

These tags are *added* at check-in (encapsulation, analogous to a sending host adding headers as data moves down the stack), *read and acted upon* at each intermediate point such as a layover/transfer airport (analogous to a router or switch processing a header at an intermediate layer), and finally *removed* – the tag is torn off at baggage claim, the boarding pass is collected at the gate – once the passenger/bag reaches its final destination (decapsulation, analogous to headers being stripped as data moves up the stack at the receiving host).